

Programmazione Avanzata

Seconda prova in itinere

14/12/2023

Note sullo svolgimento della prova

- Non è consentito l'uso di vecchi testi d'esame, libri, o appunti.
- È possibile usare solo la documentazione ufficiale delle Java API.
- Utilizzare NetBeans per editare, compilare ed eseguire i programmi.
- Chiudere tutti gli altri progetti eventualmente presenti in NetBeans.
- Creare un file **Cognome_matricola.zip** contenente i soli file *.java* da consegnare.
- Per caricare il file .zip usare il seguente form: <https://forms.gle/At3tAZ7K37jbcQN9A>
- Prima di andare via verificare con il docente che il file sia stato caricato correttamente.

Sul tavolo c'è un mazzo composto da 40 carte. Ogni carta ha un valore numerico compreso tra 1 e 10 e un seme. I semi sono: cuori, quadri, fiori e picche (in ordine di priorità decrescente). Una carta A batte una carta B se il valore numerico di A è maggiore del valore numerico di B. In caso di parità dei valori numerici si considera la priorità del seme.

Una partita è organizzata in più "giri". Il numero dei giri dipende dal numero dei giocatori. Il numero di giocatori deve essere tale da permettere un numero intero di giri.

Ad ogni giro:

- ogni giocatore, quando arriva il suo turno, pesca una carta dal mazzo (prima il giocatore 0, poi il giocatore 1, e così via).
- al termine del giro, il giocatore con la carta più alta pescata nel giro corrente ottiene un punto.

La partita termina quando sono terminate le carte del mazzo.

Realizzare il tipo enumerazione *Seme*.

Realizzare una classe *Carta* dotata almeno di un metodo

```
public boolean batte(Carta c)
```

che restituisce *true* se l'oggetto implicito batte la carta *c*, *false* altrimenti.

Realizzare una classe *Giocatore* (estensione di *Thread*) e una classe *Tavolo* che implementino il comportamento sopra descritto.

La classe *Tavolo* deve essere dotata di un costruttore

```
public Tavolo(int g)
```

che permetta di specificare il numero di giocatori. Se il valore di *g* non consente di eseguire un numero intero di giri, il costruttore lancia *IllegalArgumentException*.

Per mescolare il mazzo è possibile usare il metodo

```
static void shuffle(List<?> list)
```

della classe *java.util.Collections*. (Randomly permutes the specified list using a default source of randomness.)

Esempio di esecuzione:

```
Giocatore0: ho pescato 5C
Giocatore1: ho pescato 1Q
Giocatore2: ho pescato 7C
Giocatore3: ho pescato 4F
Giocatore4: ho pescato 8Q
Giocatore5: ho pescato 8F
Giocatore6: ho pescato 3C
Giocatore7: ho pescato 6C
Punto a Giocatore4
Giocatore0: ho pescato 3Q
Giocatore1: ho pescato 1C
Giocatore2: ho pescato 3P
Giocatore3: ho pescato 2Q
Giocatore4: ho pescato 4C
Giocatore5: ho pescato 7F
Giocatore6: ho pescato 10P
Giocatore7: ho pescato 5F
Punto a Giocatore6
Giocatore0: ho pescato 2C
Giocatore1: ho pescato 1F
Giocatore2: ho pescato 10C
Giocatore3: ho pescato 2F
Giocatore4: ho pescato 9F
Giocatore5: ho pescato 8P
Giocatore6: ho pescato 10Q
Giocatore7: ho pescato 5P
Punto a Giocatore2
Giocatore0: ho pescato 1P
Giocatore1: ho pescato 9C
Giocatore2: ho pescato 6P
Giocatore3: ho pescato 3F
Giocatore4: ho pescato 10F
Giocatore5: ho pescato 4P
Giocatore6: ho pescato 6Q
Giocatore7: ho pescato 4Q
Punto a Giocatore4
Giocatore0: ho pescato 6F
Giocatore1: ho pescato 7Q
Giocatore2: ho pescato 7P
Giocatore3: ho pescato 2P
Giocatore4: ho pescato 8C
Giocatore5: ho pescato 9P
Giocatore6: ho pescato 9Q
Giocatore7: ho pescato 5Q
Punto a Giocatore6
Giocatore0: partita terminata, ho fatto 0 punti
Giocatore2: partita terminata, ho fatto 1 punti
Giocatore6: partita terminata, ho fatto 2 punti
Giocatore1: partita terminata, ho fatto 0 punti
Giocatore3: partita terminata, ho fatto 0 punti
Giocatore5: partita terminata, ho fatto 0 punti
Giocatore4: partita terminata, ho fatto 2 punti
Giocatore7: partita terminata, ho fatto 0 punti
```